

# Understanding 2D Prefix Sums

A Visual Guide to the Inclusion-Exclusion Principle

**Explain it like I'm 5:** Imagine a large grid of rooms, and each room has some coins on the floor. If you want to know the total coins in a specific block of rooms, counting them one by one every time takes too long! Instead, we keep a "cheat sheet" (the prefix matrix). Each room in the cheat sheet tells you the total coins from the very top-left room all the way down to that specific room. Using this cheat sheet, we can find the coins in ANY block of rooms using just simple math!

## Part 1: Building the Prefix Matrix

Let's look at how your code builds the `prefix[i][j]` matrix. To find the sum of all elements from  $(0,0)$  down to  $(i,j)$ , we combine regions we've already calculated.

Original Matrix

|   |   |   |   |
|---|---|---|---|
| 3 | 0 | 1 | 4 |
| 5 | 6 | 3 | 2 |
| 1 | 2 | 0 | 1 |
| 4 | 1 | 0 | 1 |

Target: Index  $(2, 2)$



Prefix Matrix Built

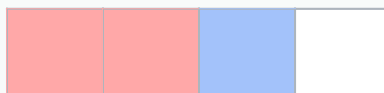
|    |    |    |    |
|----|----|----|----|
| 3  | 3  | 4  | 8  |
| 8  | 14 | 18 | 24 |
| 9  | 17 | 21 | 0  |
| 13 | 0  | 0  | 0  |

Result at  $(2, 2)$  is 21

## The Formula Visualized

Your code does this step inside the nested loop:

$$\text{Prefix}[i][j] = \text{Matrix}[i][j] + \text{Prefix}[i-1][j] + \text{Prefix}[i][j-1] - \text{Prefix}[i-1][j-1]$$





|                                                                                                          |                                                                                                   |                                                                                                    |                                                                                                             |
|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
|  <b>Current Element</b> |  <b>Top Area</b> |  <b>Left Area</b> |  <b>Top-Left Overlap</b> |
| <code>matrix[i][j]</code>                                                                                | <code>prefix[i-1][j]</code>                                                                       | <code>prefix[i][j-1]</code>                                                                        | <code>prefix[i-1][j-1]</code>                                                                               |

**Why do we subtract the Top-Left (Red) area?** When we add the Top Area (Blue) and the Left Area (Green), both of those areas *include* the Top-Left Area. Because we added it twice, we must subtract it once so it's only counted a single time.

## Part 2: Querying a Region (sumRegion)

Now, let's use the cheat sheet to find the sum of a specific region marked by (row1, col1) and (row2, col2).

Target Region: row1=1, col1=1 to row2=2, col2=2

|   |   |   |   |
|---|---|---|---|
| 3 | 0 | 1 | 4 |
| 5 | 6 | 3 | 2 |
| 1 | 2 | 0 | 1 |
| 4 | 1 | 0 | 1 |

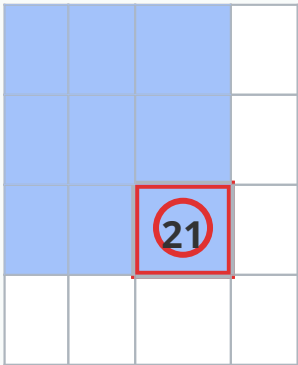
We want the sum of this yellow box (6 + 3 + 2 + 0 = 11).

### The Query Formula Visualized

Instead of looping through the yellow box, your code uses 4 lookups in the prefix matrix:

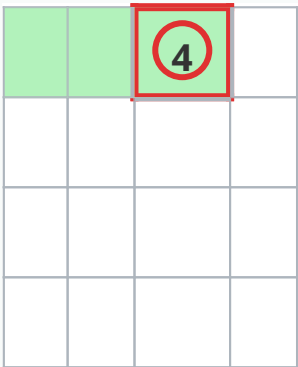
$$\text{Result} = \text{Prefix}[\text{row2}][\text{col2}] - \text{Prefix}[\text{row1}-1][\text{col2}] - \text{Prefix}[\text{row2}][\text{col1}-1] + \text{Prefix}[\text{row1}-1][\text{col1}-1]$$

1. Start with Total  
prefix[r2][c2]



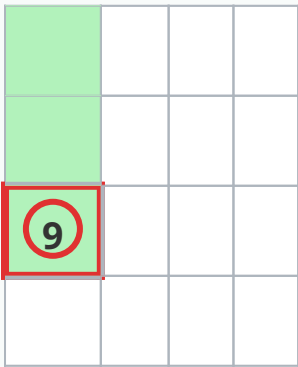
Take everything from (0,0) to the bottom-right corner.

2. Subtract Top  
- prefix[r1-1][c2]



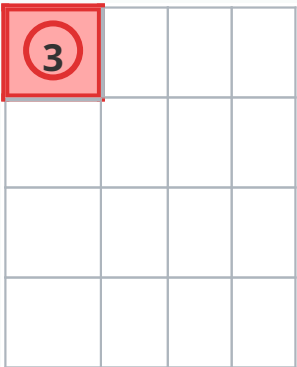
Cut off the extra rows completely above our target.

3. Subtract Left  
- prefix[r2][c1-1]



Cut off the extra columns to the left of our target.

4. Add Back Overlap  
+ prefix[r1-1][c1-1]



Top-Left was subtracted TWICE in

steps 2 and 3. Add it  
back!

**Math Check:**  $21 \text{ (Total)} - 4 \text{ (Top)} - 9 \text{ (Left)} + 3 \text{ (Overlap)} = 11$

**Summary of your code's conditions:**

Notice your code has `if (row1 > 0)` and `if (col1 > 0)`. This is simply to prevent an **Index Out of Bounds** error! If your target region touches the very top or the very left edge of the grid, there is no "Top" or "Left" piece to subtract, so the code safely skips those subtractions.

# 2D Prefix Sum: Complete Dry Run

Step-by-step trace of building the matrix and querying regions

## 1. The Target: A Simple 3x3 Matrix

Let's use a small 3x3 matrix to perform a full dry run. We want to build the prefix array from this original matrix:

Original `matrix`

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |



Final `prefix` Matrix

|    |    |    |
|----|----|----|
| 1  | 3  | 6  |
| 5  | 12 | 21 |
| 12 | 27 | 45 |

## 2. Dry Run: Building the Prefix Matrix

Your code uses this core logic to build the matrix. Let's trace it cell by cell:

```
for(int i = 0 ; i < n ; i++){
    for( int j = 0 ; j < m ; j++){
        prefix[i][j] = matrix[i][j];
        if(i > 0) prefix[i][j] += prefix[i - 1][j];           // add Top
        if(j > 0) prefix[i][j] += prefix[i][j - 1];         // add Left
        if(i > 0 && j > 0) prefix[i][j] -= prefix[i-1][j-1]; // subtract Overlap
    }
}
```

| Step (i, j) | Matrix Val | Add Top (i-1, j) | Add Left (i, j-1) | Sub Overlap (i-1, j-1) | Calculation  | Prefix Result |
|-------------|------------|------------------|-------------------|------------------------|--------------|---------------|
| (0, 0)      | 1          | None (i=0)       | None (j=0)        | None                   | <b>1</b>     | <b>1</b>      |
| (0, 1)      | 2          | None (i=0)       | prefix[0][0] = 1  | None                   | <b>2 + 1</b> | <b>3</b>      |
| (0, 2)      | 3          | None (i=0)       |                   | None                   | <b>3 + 3</b> | <b>6</b>      |

| Step (i, j) | Matrix Val | Add Top (i-1, j)  | Add Left (i, j-1) | Sub Overlap (i-1, j-1) | Calculation             | Prefix Result |
|-------------|------------|-------------------|-------------------|------------------------|-------------------------|---------------|
|             |            |                   | prefix[0][1] = 3  |                        |                         |               |
| (1, 0)      | 4          | prefix[0][0] = 1  | None (j=0)        | None                   | <b>4 + 1</b>            | <b>5</b>      |
| (1, 1)      | 5          | prefix[0][1] = 3  | prefix[1][0] = 5  | prefix[0][0] = 1       | <b>5 + 3 + 5 - 1</b>    | <b>12</b>     |
| (1, 2)      | 6          | prefix[0][2] = 6  | prefix[1][1] = 12 | prefix[0][1] = 3       | <b>6 + 6 + 12 - 3</b>   | <b>21</b>     |
| (2, 0)      | 7          | prefix[1][0] = 5  | None (j=0)        | None                   | <b>7 + 5</b>            | <b>12</b>     |
| (2, 1)      | 8          | prefix[1][1] = 12 | prefix[2][0] = 12 | prefix[1][0] = 5       | <b>8 + 12 + 12 - 5</b>  | <b>27</b>     |
| (2, 2)      | 9          | prefix[1][2] = 21 | prefix[2][1] = 27 | prefix[1][1] = 12      | <b>9 + 21 + 27 - 12</b> | <b>45</b>     |

*\*Notice how for elements in the middle like (1,1) and (2,2), the formula utilizes all 4 parts to compute the area correctly without double counting!*

### 3. Dry Run: Querying a Region sumRegion

Let's use our newly built `prefix` matrix to calculate the sum of a specific region.

**Query Goal:** Find the sum of the bottom-right 2x2 square.

**Input:** `sumRegion(row1 = 1, col1 = 1, row2 = 2, col2 = 2)`

Visualizing the Target in Original

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

Target Sum = 5 + 6 + 8 + 9 = **28**

Where to Look in Prefix

|    |    |    |
|----|----|----|
| 1  | 3  | 6  |
| 5  | 12 | 21 |
| 12 | 27 | 45 |

Accessing the boundary corners

#### Code Execution Trace for `sumRegion(1, 1, 2, 2)`

```
int res = prefix[row2][col2];           // prefix[2][2]
if(row1 > 0) res -= prefix[row1 - 1][col2]; // prefix[0][2]
if(col1 > 0) res -= prefix[row2][col1 - 1]; // prefix[2][0]
if(row1 > 0 && col1 > 0) res += prefix[row1 - 1][col1 - 1]; // prefix[0][0]
```

| Step                | Operation in Code                | Value from Prefix Array                   | Current `res` Value |
|---------------------|----------------------------------|-------------------------------------------|---------------------|
| 1. Get Total Area   | <code>res = prefix[2][2]</code>  | 45 (Sum of entire 3x3 matrix)             | 45                  |
| 2. Subtract Top     | <code>res -= prefix[0][2]</code> | 6 (Sum of Row 0: 1+2+3)                   | 45 - 6 = 39         |
| 3. Subtract Left    | <code>res -= prefix[2][0]</code> | 12 (Sum of Col 0: 1+4+7)                  | 39 - 12 = 27        |
| 4. Add Back Overlap | <code>res += prefix[0][0]</code> | 1 (The top-left cell we subtracted twice) | 27 + 1 = 28         |

**Final Result Returned: 28**

(Matches our manual calculation of  $5+6+8+9$  exactly!)